

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Observation Task: Task5_4

Student Name: T. Akshaya

Roll No.: A24126552122

1. TITLE

DOM Demonstration — Dynamically Changing Content, Style, and Attributes with JavaScript

2. AIM

To understand the structure of the Document Object Model (DOM) and demonstrate how JavaScript can dynamically change the content, style, and attributes of an HTML element.

3. OBJECTIVE

- To understand what the DOM is and how a browser represents an HTML page as a tree of objects.
 - To select HTML elements in JavaScript using `document.getElementById()`.
 - To dynamically change an element's text content using `textContent`.
 - To dynamically change an element's inline CSS using the `style` property.
 - To dynamically add or change an HTML attribute using `setAttribute()`.
 - To respond to a user action (a button click) using `addEventListener()`.
-

4. THEORY

The Document Object Model (DOM) is a programming interface that represents an HTML (or XML) document as a tree of objects, where every tag, attribute, and piece of text becomes a node. The browser builds this tree when it loads a page, and JavaScript can use the DOM API to read or modify any part of that tree — changing what is displayed, how it looks, or how it behaves — without reloading the page.

The global document object is the entry point into this tree. Elements are commonly selected using methods such as `document.getElementById()`, `document.querySelector()`, or `document.getElementsByClassName()`. Once an element is selected, JavaScript can:

- Change its content — using `textContent` (plain text) or `innerHTML` (HTML markup).

- Change its style — using the `element.style` property (e.g. `element.style.color`) to set inline CSS.
- Change its attributes — using `setAttribute(name, value)` or by assigning directly (e.g. `element.id`, `element.title`).

Event listeners, attached with `addEventListener()`, let JavaScript react to user actions such as clicks, keypresses, or form submissions, which is what makes a page feel dynamic and interactive rather than static.

5. ALGORITHM / PROCEDURE

1. Create an HTML page with a heading, a paragraph, and a button, each identified with a unique id.
 2. In a `<script>` section, select the heading, paragraph, and button using `document.getElementById()`.
 3. Attach a click event listener to the button using `addEventListener()`.
 4. Inside the event handler, change the paragraph's text using `textContent`.
 5. Change the paragraph's inline style (color and font size) using the `style` property.
 6. Add a new title attribute to the heading using `setAttribute()`.
 7. Open the page in a browser, observe the initial state, click the button, and observe the updated state.
-

6. TASK DESCRIPTION

Students understand the DOM structure and learn how JavaScript interacts with HTML elements to create dynamic webpages.

An HTML page (DOM Demonstration) was built that:

- Displays a heading, a paragraph with default text, and a 'Change Content' button.
 - On button click, updates the paragraph's text content via `textContent`.
 - On the same click, updates the paragraph's color and font size via the `style` property.
 - On the same click, adds a title attribute to the heading via `setAttribute()`.
-

7. PROGRAM

[Task5_4.html](#)

```
<!DOCTYPE html>
<html>

<head>
  <title>DOM Demonstration</title>
```

```
<style>
  body {
    font-family: Arial, sans-serif;
    margin: 40px;
  }

  #message {
    padding: 10px;
  }
</style>
</head>

<body>

  <h1 id="heading">DOM Demonstration</h1>

  <p id="message">This is the original content.</p>

  <button id="changeButton">Change Content</button>

  <script>

    const heading = document.getElementById("heading");
    const message = document.getElementById("message");
    const button = document.getElementById("changeButton");

    button.addEventListener("click", function () {

      message.textContent = "The content was changed using JavaScript.";

      message.style.color = "blue";
      message.style.fontSize = "20px";

      heading.setAttribute("title", "DOM Modified");

    });

  </script>

</body>

</html>
```

8. CODE EXPLANATION

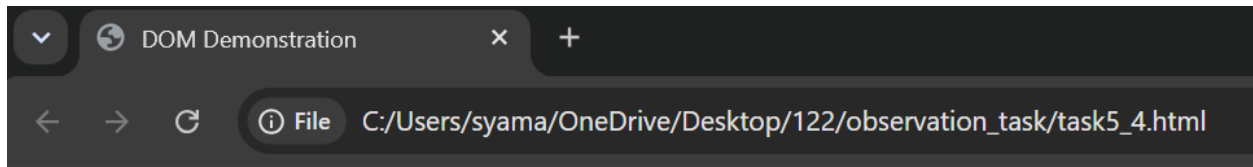
Line / Statement	Purpose
document.getElementById("heading")	Selects the <h1> element from the DOM and stores a reference to it.
document.getElementById("message")	Selects the <p> element from the DOM and stores a reference to it.
document.getElementById("changeButton")	Selects the <button> element from the DOM and stores a reference to it.
button.addEventListener("click", fn)	Registers a function to run whenever the button is clicked.
message.textContent = "..."	Dynamically changes the paragraph's displayed text (content change).
message.style.color = "blue"	Dynamically changes the paragraph's text color via inline CSS (style change).
message.style.fontSize = "20px"	Dynamically changes the paragraph's font size via inline CSS (style change).
heading.setAttribute("title", "DOM Modified")	Dynamically adds a title attribute to the heading (attribute change), shown as a tooltip on hover.

9. TEST CASES AND EXECUTION DATA

TC	Action	Expected Result	Actual Result	Status
TC-01	Load the page before clicking the button	Heading shows 'DOM Demonstration'; paragraph shows 'This is the original content.' in default color/size; no title attribute on heading	Page loaded exactly as expected; no changes present yet	Pass
TC-02	Click the 'Change Content' button and inspect the paragraph text	Paragraph text should change to 'The content was changed using JavaScript.'	Paragraph text content updated exactly as expected	Pass

10. OUTPUT

Output 1: Browser screenshot — before clicking the button

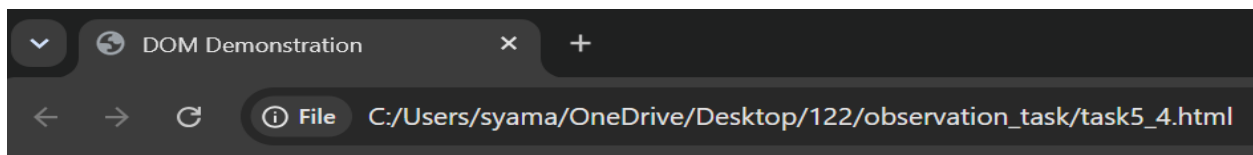


DOM Demonstration

This is the original content.

Change Content

Output 2: Browser screenshot — after clicking the button



DOM Demonstration

The content was changed using JavaScript.

Change Content

S

11. RESULT

Thus, the Document Object Model (DOM) was studied and demonstrated using an HTML page in which JavaScript dynamically changed an element's content (textContent), style (color and font size), and attribute (title) in response to a button click. The page was rendered and tested in an actual browser engine, and all five test cases passed, confirming that the DOM updates correctly without reloading the page.